

Связка Android-клиента с Hermes Agent по **WebSocket** — это оптимальный выбор для диалоговых систем, голосовых агентов и управления событиями в реальном времени.

В отличие от REST API, где клиент постоянно отправляет новые запросы, WebSocket поддерживает **одно открытое TCP-соединение**. Это обеспечивает минимальную задержку (low latency), двусторонний обмен данными и позволяет получать ответов от агента в режиме реального времени (streaming).

## 1. Архитектура подключения

[ Android App ] <--- ( Full-Duplex WebSocket ) ---> [ Hermes Gateway ]  
(Kotlin/Ktor) ws://<ip>:8642/ws (OpenAI-compatible)

1. **Инициализация:** Клиент открывает соединение по адресу `ws://<your-hermes-host>:8642/ws` (или `wss://` при наличии SSL) и передает API-ключ/токен сессии в заголовках или первом сообщении.
2. **Отправка данных (Client → Agent):** Текстовые инструкции, команды управления, аудио-чанки (при голосовом вводе) или JSON-структуры с системным контекстом.
3. **Получение ответов (Agent → Android):** Потокковая генерация текста (токен за токеном), статусы выполнения навыков (Skills/Tools) и системные уведомления.

## 2. Реализация на Android (Kotlin + Ktor Client)

Для работы с WebSocket в современный стек Android отлично подходит **Ktor Client** (или `OkHttp`).

Подключение зависимостей ( `build.gradle.kts` ):

Kotlin



```
implementation("io.ktor:ktor-client-core:2.3.12")
implementation("io.ktor:ktor-client-cio:2.3.12")
implementation("io.ktor:ktor-client-websockets:2.3.12")
```

## Класс WebSocket-сервиса:

Kotlin



```
import io.ktor.client.*
import io.ktor.client.engine.cio.*
import io.ktor.client.plugins.websocket.*
import io.ktor.http.*
import io.ktor.websocket.*
import kotlinx.coroutines.flow.*
import kotlinx.serialization.json.*

class HermesWebSocketClient(
    private val host: String,
    private val port: Int,
    private val apiKey: String
) {
    private val client = HttpClient(CIO) {
        install(WebSockets)
    }

    private var session: DefaultClientWebSocketSession? = null

    suspend fun connect(sessionId: String) {
        client.webSocket(
            method = HttpMethod.Get,
            host = host,
            port = port,
            path = "/v1/ws/chat?session_id=$sessionId",
            request = {
                header("Authorization", "Bearer $apiKey")
            }
        ) {
            session = this
            // Фоновое чтение входящих сообщений
            for (frame in incoming) {
                frame as? Frame.Text ?: continue
                val text = frame.readText()
                handleIncomingMessage(text)
            }
        }
    }

    suspend fun sendMessage(prompt: String) {
        val payload = Json.encodeToString(
            JsonObject.serializer(),
            buildJsonObject {
                put("type", "user_message")
            }
        )
    }
}
```

```

        put("content", prompt)
    }
)
session?.send(Frame.Text(payload))
}

private fun handleIncomingMessage(textJson: String) {
    // Парсинг ответов от Hermes (stream tokens, tool calls, errors)
    println("Hermes response: $textJson")
}

suspend fun close() {
    session?.close()
    client.close()
}
}

```

### 3. Формат сообщений и протокол

Обмен сообщениями происходит в формате JSON.

Запрос от Android-клиента:

JSON



```

{
  "event": "message",
  "data": {
    "role": "user",
    "content": "Включи светильник и покажи текущий статус системы"
  }
}

```

Потоковый ответ от Hermes Agent:

JSON



```

// Кадр 1 (Начало генерации/Токен)
{ "event": "token", "delta": "Выполняю" }

```

```
// Кадр 2 (Следующий токен)
{ "event": "token", "delta": " команду..." }

// Кадр 3 (Вызов инструмента/Skill)
{ "event": "tool_call", "name": "toggle_light", "args": { "state": "on" } }

// Кадр 4 (Завершение ответа)
{ "event": "done", "session_id": "session_12345" }
```

## 4. Практические рекомендации для Android

### 1. Фоновый режим и Foreground Service:

Если приложение должно оставаться на связи с Hermes при свернутом экране или при работе по Bluetooth (например, обработка медиа-кнопок гарнитуры), оберните WebSocket-клиент в **Android Foreground Service** с удерживанием `WakeLock` .

### 2. Автоматический Reconnect (Переподключение):

При смене сети (Wi-Fi ↔ Mobile Data) WebSocket-соединение разрывается. Реализуйте алгоритм *Exponential Backoff* для автоматического повторного подключения:

Kotlin



```
suspend fun connectWithRetry() {
    var delayMs = 1000L
    while (true) {
        try {
            connect(currentSessionId)
        } catch (e: Exception) {
            delay(delayMs)
            delayMs = (delayMs * 2).coerceAtMost(30000L) // Максимум 30 сек
        }
    }
}
```

### 3. Безопасность:

Если Hermes запущен в публичном облаке (VPS), обязательно используйте защищенный протокол `wss://` за Nginx/Cloudflare прокси или подключайтесь через защищенную оверлейную сеть (Tailscale / WireGuard).

